

Область видимости переменных

- Введение
- Переменные в блоках циклов и условных конструкций
- Локальные переменные
- Статические переменные
- Глобальные переменные
- Константы
- Передача аргументов по ссылке и по значению
- Время жизни переменной
- Unset внутри функции
- Некоторые итоги

Введение

Область видимости переменной — это **контекст**, в котором эта **переменная определена** (видима). В большинстве случаев все переменные PHP имеют только одну область видимости.

Важно. Единая область видимости охватывает также **включаемые** (include) и **требуемые** (require) файлы.

Например:

```
<?php  
  
    $var = 1;  
  
    include 'file.inc';
```

Здесь **переменная \$var будет доступна внутри включённого скрипта file.inc.**

Переменные в блоках циклов и условных конструкций

Блоки циклов и условных конструкций **не образуют отдельной области видимости**, и переменные, определенные в этих блоках, мы можем использовать **вне этих блоков**:

```
<?php

$condition = true;

if($condition){

    $name = "Venus";

}

echo $name; // $name доступна за пределами блока if
```

или

```
<?php

$i = 1;

while($i<10) {

    $i = $i +1;

}

echo $i; // $i доступна за пределами цикла
```

Локальные переменные

Определение (тело) пользовательской функции задаёт **локальную область видимости** данной функции.

Важно. Любая используемая **внутри функции** переменная по умолчанию ограничена **локальной областью видимости функции**.

К таким переменным можно обратиться только **изнутри** данной функции. Например:

```
<?php

function showName() {

    $name = "Venus";

    echo '$name внутри функции: ', $name, "<p>";

}

showName();

// так написать нельзя

echo '$name снаружи функции: ', $name;

// переменная $name существует только внутри функции
```

В данном случае в функции showName() определена локальная переменная **\$name**. Соответственно обратиться к этой переменной мы можем **только внутри функции showName()**, но не вне ее.

То же самое относится и к параметрам функции: **вне функции ее параметры также не существуют**.

```
<?php

function getUser($par1, $par2){

    $par1 = "admin";

    $par2 = "12345";

}

$login = "";

$pwd = "";

getUser($login, $pwd);

echo "Логин = $par1; Пароль = $par2";
```

Статические переменные

Статические переменные похожи на локальные. Их особенность в том, что после завершения работы функции их **значение сохраняется**.

Важно. При каждом новом вызове - **функция использует ранее сохраненное значение статической переменной**.

Чтобы указать, что переменная будет статической, к ней добавляется ключевое слово **static**.

example_1. Статическая переменная

```
<?php

function getCounter() {

    // объявление статической переменной

    static $counter = 0;

    $counter++;

    echo $counter;

    echo "<br />";

}

// первый вызов
getCounter(); // counter=1

// второй вызов
getCounter(); // counter=2

// третий вызов
getCounter(); // counter=3

?>
```

При трех последовательных вызовах функции **getCounter()** переменная **\$counter** будет **увеличиваться на единицу**.

Если бы переменная `$counter` была обычной нестатической, то при каждом вызове функция `getCounter()` выводила бы 1 (можно поэкспериментировать).

Как правило, статические переменные служат для создания различных счетчиков, как в примере выше.

Глобальные переменные

Глобальные переменные по умолчанию недоступны внутри функции. Например:

```
<?php

function hello(){

    echo "Hello, " . $name;

}

$name = "Tom";

hello(); // Hello,
```

Данный код **не будет работать**, а интерпретатор PHP известит нас, что переменная `$name` **не определена**.

Тем не менее, мы можем обратиться внутри функции к глобальной переменной. Для этого необходимо использовать ключевое слово **global**.

```
<?php

function hello() {

    global $name;

    echo "Hello, " . $name;

}

$name = "Tom";

hello(); // Hello, Tom
```

Для получения доступа к глобальной переменной в функции **с помощью оператора `global`** объявляется **переменная с тем же именем**:

```
global $name;
```

После этого к глобальной переменной `$name` можно будет обращаться внутри функции. При этом мы можем не только **получить** ее значение, но и **изменить** его.

example_2. Глобальная переменная

```
<?php

    function changeName () {

        global $name;

        $name = "Tomas";

    };

    $name = "Tom";

    echo $name, "<br />"; // Tom

    changeName ();

    echo $name; // Tomas

?>
```

Константы

Константы — это именованные значения, которые не могут быть изменены во время выполнения программы. Они отличаются от переменных тем, что их значение остаётся постоянным на протяжении всего скрипта

Важно. Константы в PHP доступны во всех областях видимости, включая **глобальную** и **локальную** области видимости.

Константы могут содержать только **скалярные значения** (строки, числа или булевы значения) или **массивы**.

Для определения константы в PHP используется функция **define()**.

Константы в PHP необходимо определить перед их использованием.

example_3. Константа в глобальной области видимости

```
<?php

// объявление константы в глобальной области видимости
define('MYCONST', "Глядя на мир, нельзя не удивляться!");

// объявление глобальной переменной
$cite = "Нельзя объять необъятное!";

function outVariable() {

    // вывод константы

    echo '<h3>' . MYCONST . '</h3>';

    // вывод переменной

    echo '<h3>' . $cite . '</h3>';

};

outVariable();

?>
```

example_4. Константа в локальной области видимости

```
<?php

// функция определения константы

function outVariable() {

    // объявление константы в локальной области видимости

    define('MYCONST', "Глядя на мир, нельзя не удивляться!");

};
```

```
// вызов функции

outVariable();

// вывод константы, определенной в функции

echo '<h3>' . MYCONST . '</h3>';

?>
```

Передача аргументов по ссылке и по значению

По умолчанию аргументы функции передаются **по значению**. Это означает, что если значение аргумента внутри функции изменяется, оно **не изменится вне функции**.

example_5. Передача аргумента по значению

```
<?php

// передаем аргумент по значению

function inc($par) {

    $par = $par + 10;

    echo "Локальная переменная = " . $par; // 15

    echo "<p>";

}

$var = 5;

// вызываем функцию

inc($var);

echo "Глобальная переменная = " . $var; // 5

?>
```

Чтобы функция могла изменять свои аргументы, они должны **передаваться по ссылке**. Чтобы аргумент функции передавался по

ссылке, необходимо добавить амперсанд (&) к имени параметра в определении функции.

К сведению. На самом деле запомнить это правило не трудно. **По значению** - значит, передается копия значения переменной. **По ссылке** - передается ссылка на саму переменную.

В следующем листинге приведен пример передачи аргумента по ссылке.

example_6. Передача аргумента по ссылке

```
<?php

    // передаем аргумент по ссылке

    function inc(&$par) {

        $par = $par + 10;

        echo "Локальная переменная = " . $par; // 15

        echo "<p>";

    }

    $var = 5;

    // вызываем функцию

    inc($var);

    echo "Глобальная переменная = " . $var; // 15

?>
```

В вызове функции знак ссылки указывать не нужно, он есть только в определении функции. Этого достаточно для корректной передачи аргументов по ссылке.

При передаче в качестве аргументов функции массивов необходимо придерживаться тех же правил. Ниже представлены примеры передачи массива по значению и по ссылке.

example_7. Массив как аргумент по значению

```
<?php

// передаем аргумент-массив по значению

function fnChangingArray($arr) {

    $arr[0] = "javascript";

    $arr[1] = "css";

    $arr[2] = "vue.js";

}

$array = array("php", "laravel", "mysql");

echo "<pre>";

print_r($array);

// вызываем функцию

fnChangingArray($array);

echo "<pre>";

print_r($array);

?>
```

example_8. Массив как аргумент по ссылке

```
<?php

// передаем аргумент-массив по ссылке

function fnChangingArray(&$arr) {

    $arr[0] = "javascript";

    $arr[1] = "css";

    $arr[2] = "vue.js";

}

$array = array("php", "laravel", "mysql");

echo "<pre>";
```

```
print_r($array);

// вызываем функцию
fnChangingArray($array);

echo "<p>";

print_r($array);

?>
```

Важно. Исключение в языке программирования PHP составляют объекты. **Объекты всегда передаются по ссылке.**

Пример передачи объекта в виде аргумента функции представлен в листинге. Если действия с объектом вам непонятны, ничего страшного, главное убедиться, что свойства объекта (role, pwd) меняются. Объекты будем изучать далее.

example_9. Объект как аргумент по значению

```
<?php

// создаем класс
class UserClass {

    public $login = "master";

    public $role = "user";

    public $pwd = "12345";

}

// передаем объект в качестве аргумента по значению
function fnChangingObj($obj) {

    global $role, $pwd;

    $obj->role = $role;

    $obj->pwd = $pwd;
```

```
}

// создаем экземпляр класса

$user = new UserClass();

echo "<pre>";

print_r($user);

$role = "admin";

$pwd = "qwerty";

fnChangingObj($user);

echo "<p>";

print_r($user);

?>
```

К сведению. На самом деле с объектами все **немного сложнее**. И знак амперсанда (&) в сигнатуре функции все таки имеет значение. Подробнее этот вопрос будет рассмотрен в теме, посвященной объектам.

Время жизни переменной

Временем жизни переменной называется интервал выполнения программы, в течение которого она существует.

Важно. Интервал выполнения программы в котором существует переменная есть **время жизни** этой переменной.

Поскольку локальные переменные имеют своей областью видимости функцию, то время жизни локальной переменной определяется временем выполнения функции, в которой она объявлена. Это означает,

что в разных функциях совершенно независимо друг от друга могут использоваться переменные с одинаковыми именами.

Локальная переменная при каждом вызове функции **инициализируется заново**, поэтому функция-счетчик, в приведенном ниже примере всегда будет возвращать значение 1:

```
<?php

function counter() {

    $counter = 0;

    return ++$counter;

}
```

Для того, чтобы локальная переменная сохраняла свое предыдущее значение при новых вызовах функции, ее можно объявить статической при помощи ключевого слова **static**:

```
<?php

function counter() {

    static $counter = 0;

    return ++$counter;

}
```

Важно. Временем жизни статических переменных является **время выполнения сценария**.

Т. е., если пользователь перезагружает страницу, что приводит к новому выполнению сценария, переменная **\$counter** в этом случае инициализируется заново.

Unset внутри функции

Ранее мы рассмотрели функцию **unset()** — удаляющую одну или несколько перечисленных переменных.

```
<?php

$var = 'global variable';

var_dump($var);

echo "<p>";

unset($var);

var_dump($var);
```

Поведение **unset()** **внутри пользовательской функции** может отличаться, в зависимости от того, какой тип имеет переменная, которую необходимо удалить.

Если переменная, объявленная **глобальной**, удаляется внутри функции, то будет удалена только **локальная переменная**. Переменная в области видимости вызова функции сохранит то же значение, что и до вызова **unset()**.

example 10. Поведение unset при удалении глобальной переменной

```
<?php

function fnUnsetVar() {

    // объявляем переменную глобальной
    global $var;

    // удаляем глобальную переменную внутри функции
    unset($var);

    var_dump($var); // Warning: Undefined variable $var
    echo "<p>";

}
```

```

// иницилируем глобальную переменную

$var = 'global variable';

fnUnsetVar();

// выводим глобальную переменную

// во внешней программе ничего с переменной не произошло

var_dump($var);

?>

```

Если переменная, которая передаётся **по ссылке**, удаляется внутри функции, то будет удалена только **локальная переменная**.

Переменная в области видимости вызова функции сохранит то же значение, что и до вызова **unset()**.

example_11. Поведение unset при удалении переменной по ссылке

```

<?php

function fnUnsetVar(&$par) {

    // удаляем переменную переданную по ссылке

    unset($par);

    var_dump($par); // Warning: Undefined variable $par

    echo "<p>";

}

// иницилируем глобальную переменную

$var = 'global variable';

fnUnsetVar($var);

// выводим глобальную переменную

// во внешней программе ничего с переменной не произошло

var_dump($var);

?>

```

Некоторые итоги

Подведем некоторые итоги чрезвычайно важной теме, посвященной функциям.

Функции это **именованный блок кода**, который можно многократно вызывать в различных частях программы. Это своего рода "переменная", в которой хранится блок кода, который **выполняется** при использовании этой "переменной".

Функции позволяют разделять программу на меньшие **функциональные** части. Это очень мощный инструмент **повторного использования кода**. Чтобы упростить себе работу, мы можем оформить в виде функции некоторую часть кода, которая используется в сценарии несколько раз.

Функция не выполняется сразу при загрузке страницы. Функция будет выполняться при ее вызове из тела программы.

Разделяют два типа функций — **встроенные** и **пользовательские**.

- **Встроенные функции** — это функции, которые за нас уже написали создатели языка программирования, и мы можем просто брать их и использовать. В PHP существуют тысячи готовых функций.
- **Пользовательские функции** — это функции, которые программист создает самостоятельно.

Функции могут принимать на вход некоторую информацию из внешней программы. **Информация** передается в функцию через **параметры**. При этом следует различать следующие понятия:

- **Параметр** — **переменная**, создаваемая при определении функции.

- **Аргумент — значение**, которое передается в функцию при её вызове.

Существует два **основных** способа передать функции входные параметры:

1. сделать переменную **глобальной**,
2. передать переменную **параметром** функции:
 - a. по значению,
 - b. по ссылке.

Замечание. Мы не берем в рассмотрение **суперглобальные переменные массива \$GLOBALS**, изучением которых займемся в отдельном уроке.

Закрепим знания о каждом способе еще раз в отдельных примерах.

example_12/index.php. Использование глобальных переменных

```
<?php

// включаем массив $team

// проверьте наличие файла team.php в директории

require "team.php";

// определяем функцию

function search() {

    // делаем переменные глобальными

    global $team, $name;

    foreach ($team as $key => $item) {

        if ($item["name"] == $name) {

            return $item;

        }

    }

}
```

```
}

// определяем критерий поиска

$name = "Aerosmith";

// вызываем функцию поиска

$data = search();

// выводим результат

echo "<pre>";

var_dump($data);

?>
```

example_13/index.php. Использование аргументов функции

```
<?php

// включаем массив $team

// проверьте наличие файла team.php в директории

require "team.php";

// определяем функцию с двумя параметрами

function search($arr, $term) {

    foreach ($arr as $key => $item) {

        if ($item["name"] == $term) {

            return $item;

        }

    }

}

// определяем критерий поиска

$name = "Aerosmith";

// вызываем функцию поиска

$data = search($team, $name);
```

```
// выводим результат
```

```
echo "<pre>";
```

```
var_dump($data);
```

```
?>
```